

Mikrocomputertechnik 1

Einheit 4: Funktionen & der Stack

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 61

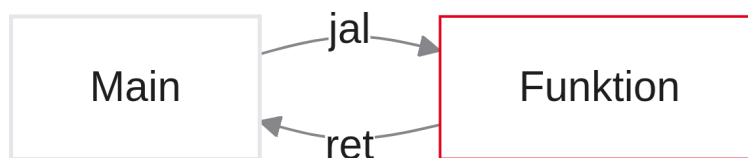
Funktionen & der Stack

Folie

2 / 61

Die Kunst der Unterprogramme

- Herzlich willkommen zur vierten Vorlesung in Mikrocomputertechnik 1
- Wir verlassen das Niveau einfacher Skripte und Spaghetti-Codes
- Ziel: wiederverwendbare, professionelle Code-Blöcke (Funktionen)
- Wie findet der Prozessor zu seinem Aufrufer zurück?
- Der Stack als sicherer Tresor für lokale Variablen und gerettete Register

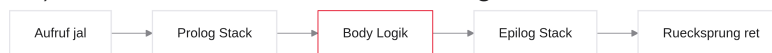


Notizen

Herzlich willkommen zu Einheit 4. Arithmetik, Speicher und Kontrollfluss beherrschen Sie. Was fehlt, ist Struktur: In C/Java lagern Sie Logik in Funktionen aus. In RISC-V teilen sich alle Funktionen 32 Register — das braucht Spielregeln (ABI) und den Stack.

Agenda für den heutigen Tag

1. Rücksprünge — warum einfache Jumps nicht reichen und das Register `ra`
2. Die ABI und der Register-Krieg — wer darf welches Register wann überschreiben?
3. Der Stack — LIFO und der Stack Pointer im Arbeitsspeicher
4. Prolog und Epilog — der standardisierte Frame für jede Funktion
5. Verschachtelung — wenn Funktionen wiederum Funktionen aufrufen
6. Praxis (Venus) — Stack-Traces und Vorbereitung der Rekursion



Notizen

Zuerst Navigation: Woher kam der Aufruf? Dann der Konflikt um Register. Danach der Stack als dynamischer Speicher für Save/Restore. Prolog/Epilog sind der Compiler-Bauplan; zum Schluss Verschachtelung und Venus. Den visuellen Datenpfad betrachten wir später in Ripes.

Rückblick: Was wir bisher können

- Bedingt (`beq`, `blt`) oder unbedingt (`j`) springen
- Pointer auf Arrays und Offsets (`lw`, `sw`)
- `while`- und `for`-Schleifen in Assembler
- Der Program Counter (PC) als Dirigent
- Invertierung für If/Else-Logik

```
# Bisher: bedingungsloser Sprung
j Ueberspringen
```

```
# ... toter Code ...
```

```
Ueberspringen:
# hier geht es weiter
```

Notizen

Den PC manipulieren können Sie sicher. Labels und `j` reichen für If und Schleifen — aber nicht für wiederverwendbare Unterprogramme mit dynamischem Rückweg.

Das Copy-Paste-Problem

- Szenario: Code-Block, der eine Zahl quadriert
- Benötigt an drei Stellen im Programm
- Bisher: Assembler-Zeilen dreimal kopieren
- Nachteil: `.text` wird groß und frisst RAM
- Bugfixen heißt: drei Stellen pflegen

```
int a = quadriere(5);  
// ... anderer Code ...  
int b = quadriere(10);  
int c = quadriere(x);
```

Notizen

Funktionen vermeiden Aufblähen und Mehrfachpflege. Einmal definieren, vielfach aufrufen — Software-Engineering auf Hardware-Ebene.

Funktionen in Hardware

- Den Quadrieren-Code nur einmal in den RAM legen
- Label z. B. `Quadriere:`
- Bei Bedarf: PC dorthin springen
- Danach zurück zum Aufrufer (`main`)
- Isolierter Block: Parameter rein, Ergebnis raus

```
Main:  
    # Springe zur Funktion  
    # <--- danach hier weiterarbeiten  
  
Quadriere:  
    # z. B. mit M-Extension: mul t0, a0, a0  
    # Zurueckspringen!
```

Notizen

Physikalisch: ein Instruktionsblock mit Label. Herausforderung: am Ende exakt in die Zeile unter dem Aufruf zurückkehren — nicht an eine fest verdrahtete Adresse.

Versuch 1: Der einfache Jump

- Aufruf: `j Quadriere` — PC springt korrekt
- Rücksprung: `j Main_Zeile_2` am Funktionsende
- Fatal: Rücksprung ist hardcodiert
- Zweiter Aufruf von `Main_Zeile_50` landet trotzdem bei Zeile 2

```
Main_Aufruf_1:
    j Quadriere
Main_Zeile_2:

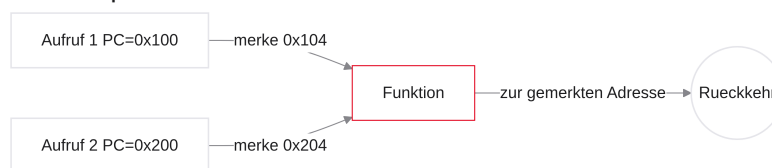
Main_Aufruf_2:
    j Quadriere
Main_Zeile_50:                # FATAL: Funktion springt immer zu Zeile 2
```

Notizen

Ein fester Rücksprung-Jump macht die Funktion unbrauchbar für mehrere Aufrufstellen. Wir brauchen einen dynamischen Rückweg.

Das „Wo komme ich her?“-Problem

- Eine echte Funktion muss blind aufrufbar sein
- Sie darf nicht im Voraus wissen, wer sie aufruft
- Im Moment des Aufrufs: aktuelle Folgeadresse merken
- Diese Brotkrumenspur heißt Return Address



Notizen

Dynamisch: Beim Sprung die Hausnummer der Folgezeile ($PC + 4$) aufschreiben. Nur so findet die Funktion den Weg zurück — unabhängig vom Aufrufer.

Die Return Address (**ra**)

- Dediziertes Spezial-Register: Architektur-Register $x1$
- ABI-Name: `ra` (Return Address)
- Speichert die Adresse der Instruktion direkt nach dem Aufruf
- Rettungsanker für den Rückweg aus dem Unterprogramm

```
# PC = 0x00400000 (Aufruf-Befehl)
# ra muss 0x00400004 werden (Zeile darunter)
```

Notizen

Ab heute Pflichtwissen: $x1/ra$. Konvention: vor/beim Betreten der Funktion liegt die Folgeadresse in `ra`. Die Funktion vertraut blind darauf.

Jump and Link (**jal**)

- Ein Befehl, zwei Ergebnisse aus dem **alten** PC
- „Ein Takt“ ist eine Annahme des Single-Cycle-Lehrmodells, keine ISA-Regel
- Linkwert = alter PC + 4
- Ziel = alter PC + Byteoffset
- `ra` ist ein normales GPR mit ABI-Rolle

```
jal ra, Quadriere
# ra = oldPC + 4
# PC = oldPC + Byteoffset
```

Notizen

`jal` = Jump and Link. Ticket nach Hause in `ra`, gleichzeitig Sprung zur Funktion. Das ist der Standard-Funktionsaufruf.

Der Funktionsaufruf

- Standard ab heute: `jal ra, Label`
- Pseudo `j` aus Einheit 3 war getarntes `jal`
- Venus übersetzt `j Label` zu `jal x0, Label`
- $PC + 4$ landet in `x0` und verpufft (schreibgeschützt)

```
# Aufruf mit Rueckkehr:
jal ra, MeineFunktion

# Simpler Sprung ohne Rueckkehr (If/Else):
j ElseBlock      # intern: jal x0, ElseBlock
```

Notizen

Kreis zu Einheit 3: Beim Else-Überspringen wird das Rückflugticket berechnet und in `x0` verworfen. Für echte Funktionen: `jal ra`, damit das Ticket in `ra` landet.

Jump and Link Register (`jalr`)

- Am Funktionsende hilft `jal` nicht (braucht festes Label)
- Ziel ist die dynamische Adresse in `ra`
- `jalr: Ziel = (rs1 + signextend(imm))` mit gelöschtem Bit 0
- Linkwert = alter PC + 4
- `ret` ist `jalr x0, 0(ra)`

```
jalr x0, 0(ra)

# PC = ra + 0
# Link-Ziel x0: keine zweite Brotkrumenspur
```

Notizen

`jalr` springt zur Zahl in einem Register. Für die Heimreise: Quelle `ra`, Link-Ziel `x0` (kein neues Ticket nötig).

Der Rücksprung (ret)

- `jalr x0, 0(ra)` ist korrekt, aber unleserlich
- Pseudo-Instruktion: `ret`
- Venus generiert daraus den `jalr`-Befehl
- Assembler-Äquivalent zu `return` in C/Java

Quadriere:

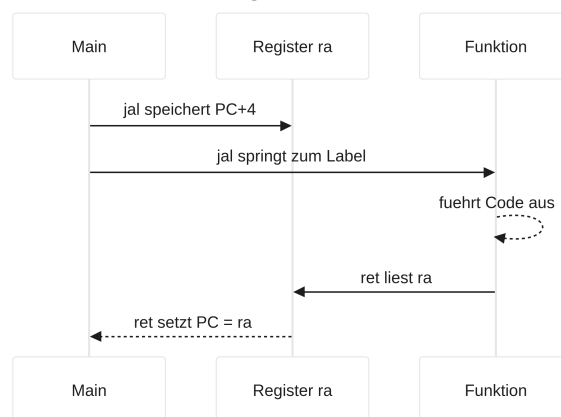
```
# ... Funktionsrumpf ...
ret           # = jalr x0, 0(ra)
```

Notizen

Am Funktionsende einfach `ret` tippen — dynamischer Sprung über `ra` im Hintergrund.

Visualisierung: Der Zyklus

- Zusammenspiel von `jal` und `ret` als geschlossener Kreis
- Brotkrumenspur beim Absprung gelegt, beim Rücksprung eingelöst
- Skaliert: dieselbe Funktion von beliebig vielen Stellen aufrufbar

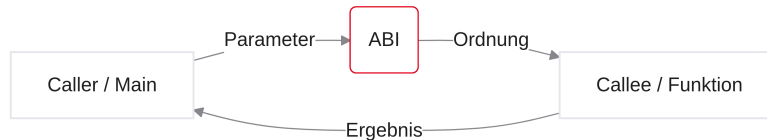


Notizen

Tanz der Register: `jal` speichert Ticket und springt; `ret` löst das Ticket ein. Die Funktion bleibt isoliert vom Aufrufer.

Die ABI und der Register-Krieg

- Funktionen wissen nichts voneinander (blinder Flug)
- Sichere Datenübergabe: a-Register
- Konflikt: 32 Register für das gesamte Programm
- Lösung: Application Binary Interface (ABI) als sozialer Vertrag
- Flüchtig (Caller-Saved) vs. sicher (Callee-Saved)



Notizen

Navigation ist gelöst. Nun Daten und Register-Krieg: Wer überschreibt `s0`? Ohne ABI würde jedes große Programm kollabieren.

Datenübergabe (Argumente)

- Parameter und Ergebnis in C: `calc(5, 10);`
- In RISC-V: vor dem Sprung Werte in Register legen
- ABI: `a0` bis `a7` für Argumente
- Die Funktion erwartet ihre Zutaten exakt dort

```
li a0, 5
li a1, 10
jal ra, Calc
```

Notizen

Briefkästen der ABI: Argumente vor `jal` in `a0–a7`. Die Funktion liest starr dort.

Der Rückgabewert

- Ergebnis (Return-Value) zwingend in a0
- Vor `ret`: Endresultat nach a0 kopieren
- Aufrufer holt die Antwort nach dem `jal` aus a0

```
Calc:
    add t0, a0, a1
    mv a0, t0           # ABI: Ergebnis in a0
    ret
```

Notizen

a0 ist der Rückgabe-Briefkasten. Optional zweites Ergebnis in a1 — selten. (mv ist Pseudo für `addi rd, rs, 0` bzw. `add` mit `x0`.)

Code-Beispiel: Parameter komplett

- Vertrag Caller/Callee in der Praxis
- Compiler-Muster hinter `int x = func(y);`

```
Main:
    li a0, 5
    li a1, 10
    jal ra, Addiere
    mv s0, a0           # Ergebnis sichern
    li a0, 10           # Venus: exit
    ecall

Addiere:
    add a0, a0, a1      # Summe direkt in a0
    ret
```

Notizen

Main bereitet Argumente vor, Addiere schreibt die Summe nach a0, Main sichert nach s0 — konventioneller ABI-Code. Ohne Exit nach dem `mv` würde der Fall-through in `Addiere` eine Endlosschleife erzeugen.

Der Register-Konflikt

- Nur 32 Architekturregister, von allen Funktionen geteilt
- Szenario: `Main` speichert Zwischenwert in `s0`, ruft `Print()` auf
- `Print()` nutzt `s0` ahnungslos als Zähler
- Nach `ret`: Wert in `Main` ist zerstört

Notizen

Lebenspunkte in `s0`, Zeichenfunktion überschreibt `s0` — nach Rückkehr ist der Spieler „tot“. Katastrophaler Logikfehler ohne sichtbare CPU-Warnung.

Zerstörte Variablen (Trace)

- Funktionen werden isoliert programmiert (Bibliotheken)
- Autor von `Print` kennt die Register von `Main` nicht
- CPU blockiert Register nicht — Schreiben überschreibt Flip-Flops

```
Main:
    li s0, 99
    jal ra, BoeseFunc
    # s0 ist jetzt 0
    li a0, 10
    ecall
```

```
BoeseFunc:
    li s0, 0
    ret
```

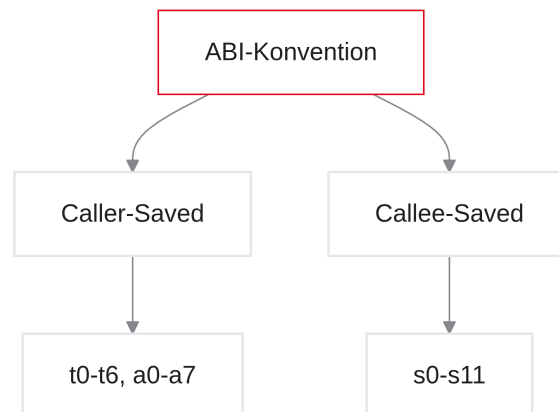
`Main` endet mit dem Venus-Exit. Es gibt keinen Fall-through in `BoeseFunc`. Der isolierte Fehler ist das ungesicherte `s0`.

Notizen

Clobbering: stille Zerstörung. Ohne strenge Konventionen ist fremder Code unsicher aufrufbar.

Die Lösung: ABI-Verträge

- Hardware sperrt keine Register — Software muss retten
- ABI erzwingt Save/Restore vor Überschreiben
- Inhalt in den RAM evakuieren, später wiederherstellen
- Offene Frage: Wer ist verantwortlich — Caller oder Callee?



Notizen

Evakuieren vor Nutzen: So wirkt das Register unverändert. Die Fraktionen Caller-Saved vs. Callee-Saved klären die Haftung.

Rollenverteilung: Caller vs. Callee

- Caller (Aufrufer): führt `jal` aus (z. B. `Main`)
- Callee (Aufgerufener): wird angesprungen und arbeitet
- Rollen sind relativ: wer selbst aufruft, wird Caller
- ABI teilt Register in zwei Haftungspflichten

```
Main:          <--- CALLER
    jal ra, Print

Print:         <--- CALLEE
    ret
```

Notizen

Wenn `Print` später `DrawPixel` aufruft, wird `Print` selbst zum Caller. Die Begriffe hängen am Aufruf, nicht am Funktionsnamen.

Caller-Saved Register (Flüchtig)

- Register: `t0–t6`, `a0–a7`, und `ra`
- ABI: Not Preserved — flüchtig
- Callee darf sie ohne Rettung überschreiben
- Caller muss wichtige Werte vor `jal` selbst sichern

```
# Caller schuetzt sich selbst:
# ... t0 in den RAM retten ...
jal ra, Funktion
# ... t0 wiederherstellen ...
```

Notizen

Freiwild für den Callee. Nach jedem `jal`: alle `t`-Register mental als gelöscht betrachten. Vertrauen Sie keinem `t0` über einen Aufruf hinweg.

Die Konsequenz für `t`-Register

- `t`-Register nur für Kurzzeit-Rechnungen
- Bei `jal`: alle `t`-Register als gelöscht betrachten
- Vorteil: kleine Hilfsfunktionen ohne teure RAM-Backups

```
/* Kein C-Beispiel: der Compiler erhält die Lebensdauer. */
```

```
# Isolierter ABI-Fehler, kein C:
li    t0, 5
jal   ra, Print      # Print darf t0 überschreiben
# t0 ist danach nicht mehr verlässlich 5
```

Notizen

Das ist Assembler, keine C-Semantik. Ein korrekter C-Compiler legt die Variable so, dass sie den Aufruf überlebt: anderes Register oder Sicherung. Ungesichertes `t0` über `jal` ist der eigentliche Fehler.

Callee-Saved Register (Sicher)

- Register: `s0–s11`
- ABI: Preserved — sicher über den Aufruf hinweg
- Caller darf sich darauf verlassen
- Callee, der `s` nutzt: alten Wert retten und vor `ret` wiederherstellen

Funktion:

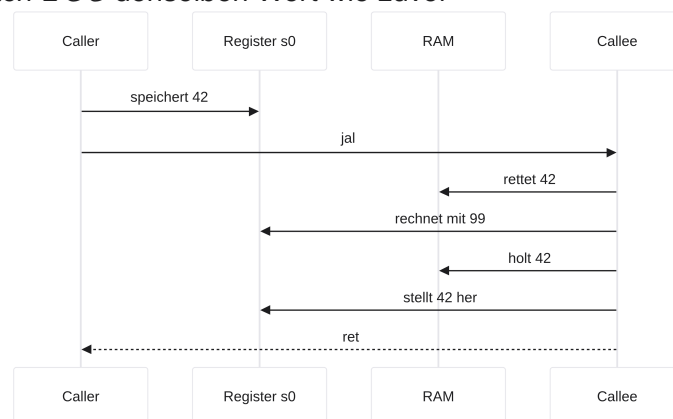
```
# 1. fremdes s0 in den RAM
# 2. eigene Rechnung mit s0
# 3. altes s0 zurueck
ret
```

Notizen

Tresor-Register. Haftung liegt beim Callee: Save — Modify — Restore. Der Caller merkt von der Zwischennutzung nichts.

Das Versprechen in der Praxis

- Callee schützt `s0` vor der Zerstörung
- Caller sieht nach `ret` denselben Wert wie zuvor



Notizen

Save-Modify-Restore: Evakuieren, nutzen, zurückschreiben — System bleibt stabil.

Das „Wo?“-Problem

- Retten geht in den RAM (nicht in ein anderes Register — Teufelskreis)
- Hartcodierte Adresse `0x1000` scheitert bei Verschachtelung/Rekursion
- Jeder Aufruf bräuchte exklusiven, frischen Speicher
- Wir brauchen einen dynamisch wachsenden Bereich

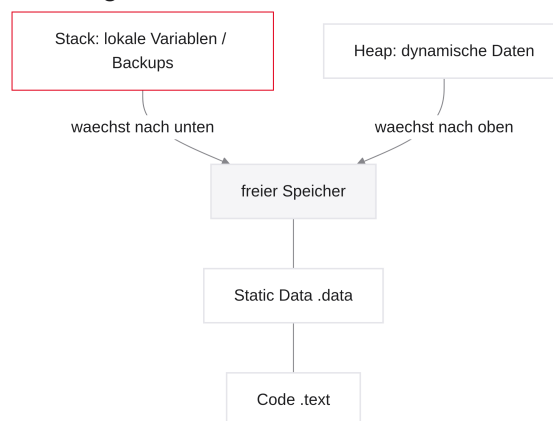
Benötigt: Ein Ort im RAM, der für JEDEN Funktionsaufruf frisch und exklusiv bereitsteht.

Notizen

Zehn rekursive Aufrufe dürfen sich nicht gegenseitig die Backups überschreiben. Lösung: der Stack — zuerst das RAM-Layout, dann LIFO und `sp`.

Die Struktur des Arbeitsspeichers (RAM)

- Wie teilt ein modernes Betriebssystem den RAM auf?
- Text / Code: ganz unten das kompilierte Maschinenprogramm
- Static Data: globale Variablen (z. B. Arrays unter `.data`)
- Heap: dynamischer Speicher (z. B. `malloc` in C), wächst nach oben
- Stack: startet oben bei der größten Adresse und wächst nach unten



Notizen

Unten: Programmcode, darüber statische Daten und Heap. Der Stack beginnt bei einer hohen Adresse und wächst nach unten. Das ist eine gewählte Belegung mit Betriebssystem. Der Stack muss den Heap nicht erreichen, bevor eine Grenze gilt.

Der Stack (LIFO und Stack Pointer)

- Dynamisch wachsender Keller für gerettete Register und lokale Variablen
- LIFO: Last In, First Out
- Stack Pointer (`sp`): Zeigefinger auf die Spitze
- Push und Pop in RISC-V-Assembler

Notizen

Das RAM-Layout kennen Sie. Nun die Mechanik: LIFO, `sp` und warum der Keller nach unten wächst.

Das Stack-Konzept (LIFO)

- Stack = abstrakte Datenstruktur (Stapel / Keller)
- Metapher: Tellerstapel in der Cafeteria
- Push: neuer Teller oben auflegen
- Pop: nur den obersten Teller nehmen
- LIFO: Last In, First Out — keine Operation in der Mitte

Notizen

Push sichert eine Variable oben auf dem Stapel; Pop holt sie wieder. Die goldene Regel: immer nur an der Spitze arbeiten.

Der Stack Pointer (`sp`)

- Zeigefinger auf die Spitze des Stapels im RAM
- Architektur-Register `x2`, ABI-Name `sp`
- `sp` hält die Adresse des zuletzt belegten Faches
- Beim Boot: Betriebssystem setzt `sp` auf eine hohe RAM-Adresse

```
# sp zeigt immer auf den aktuell obersten Eintrag
# Ablegen = sp verschieben, dann speichern
```

Notizen

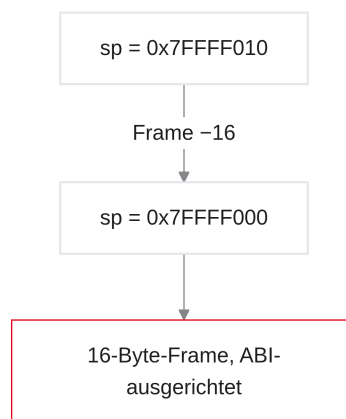
Orientieren Sie sich am `sp`: Er ist die Hausnummer der Stack-Spitze. Ohne ihn wäre Push/Pop nicht adressierbar.

Folie

32/61

RISC-V-Regel: Stack wächst nach unten

- Stack beginnt oben — Wachstum heißt Richtung Adresse 0
- Push (Platz schaffen): Adressen werden kleiner — Subtraktion von `sp`
- Pop (Platz freigeben): Adressen werden wieder größer — Addition zu `sp`
- Historisch: Kollision mit dem von unten wachsenden Heap verzögern



Notizen

Kontraintuitiv, aber Standard: nach unten wachsen heißt `sp` verkleinern zum Reservieren und vergrößern zum Freigeben.

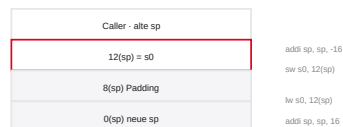
Push: Etwas ablegen (Schritt 1)

- Register `s0` vor dem Überschreiben sichern
- ABI-Konvention ILP32: der Frame ist 16 Byte groß, auch für ein Wort
- `addi sp, sp, -16`, danach `sw` innerhalb des Frames
- Padding füllt den Rest; `sp` bleibt an der Aufrufgrenze ausgerichtet

```
addi sp, sp, -16    # ein Wort plus Padding
```

ILP32-Frame, 16 Byte, ein gesichertes Register plus Padding

ABI-Konvention: `sp` vor und nach dem Aufruf durch 16 teilbar. Erst reservieren, dann schreiben.



Mehrere Register teilen sich denselben Frame, ra einer Non-Leaf-Funktion wird einmal im Prolog gesichert.

Abbildung 1: 16-Byte-Frame mit einem gesicherten Register

Notizen

Schritt 1: 16 Byte reservieren. Erst danach schreiben. Ein einzelnes Register liegt bei 12 (`sp`); die übrigen 12 Byte sind Padding. Venus akzeptiert auch kleinere Subtraktionen, der Kurs lehrt die ILP32-Ausrichtung.

Push: Etwas ablegen (Schritt 2)

- Daten mit `sw` in das reservierte Fach schreiben
- Base: der frisch verschobene `sp`, Offset 0
- Befehl: `sw s0, 0(sp)`

```
addi sp, sp, -16
sw s0, 12(sp)
```

Notizen

Kompletter Push: erst Platz, dann Store. `s0` liegt im Tresor; das Register darf im Body überschrieben werden.

Pop: Etwas herunternehmen (Schritt 1)

- Vor `ret`: `s0` aus dem RAM zurückholen
- `sp` zeigt noch auf den abgelegten Eintrag
- `lw s0, 0(sp)` — Wert wieder in der CPU
- RAM-Platz ist noch als belegt markiert

```
lw s0, 12(sp)
```

Notizen

Erst laden, dann freigeben. Nach dem `lw` liegt der Wert wieder in `s0`, der Teller liegt aber noch auf dem Stapel.

Pop: Etwas herunternehmen (Schritt 2)

- Platz freigeben: `addi sp, sp, 16`
- Bits im RAM bleiben als „Elektroschrott“, bis der nächste Push überschreibt
- `sp` steht wieder dort, wo er vor dem Push war

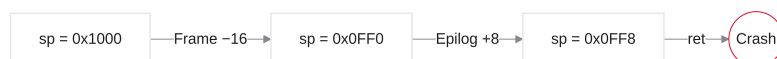
```
lw s0, 12(sp)
addi sp, sp, 16
```

Notizen

Freigabe = `sp` nach oben. Kein explizites Löschen nötig — der nächste Push überschreibt den alten Inhalt.

Das Gleichgewichts-Gesetz

- Jede Funktion hinterlässt den Stack exakt wie vorgefunden
- Summe der Pushes = Summe der Pops
- Beispiel: Prolog $-16 \rightarrow$ Epilog $+16$ vor `ret`
- Ungleichgewicht verschiebt Frames des Aufrufers — Crash



Notizen

Take nothing but pictures, leave nothing but footprints: Was Sie am Anfang vom `sp` abziehen, müssen Sie am Ende wieder addieren. Sonst liest der Caller falsche Offsets.

Folie

38/61

Effizienz: Multi-Push

- Viermal einzeln reservieren ist unnötig
- Besser: ein Frame, Vielfaches von 16 Byte
- Zwei Register passen in denselben 16-Byte-Frame

```
# Ineffizient: vier einzelne Wortreservierungen
# Kursrahmen: ein gemeinsamer 16-Byte-Frame
addi sp, sp, -16
sw s0, 12(sp)
sw s1, 8(sp)
```

Notizen

Compiler reservieren den gesamten Frame in einem Rutsch und belegen ihn mit Offsets — weniger ALU-Takte.

Folie

39/61

Array auf dem Stack (Offsets)

- Nach `addi sp, sp, -16` zeigt `sp` auf das untere Ende
- Drei Wörter plus 4 Byte Padding: Offsets 12, 8 und 4

```
addi sp, sp, -16
sw ra, 12(sp)
sw s1, 8(sp)
sw s0, 4(sp)
```

Notizen

0 (`sp`) unten, positive Offsets nach oben im Frame. Kein weiteres Verschieben von `sp` nötig, bis der Epilog den Block freigibt.

Visualisierung im RAM

- So sieht ein Multi-Push im Venus-Memory-Tab aus:

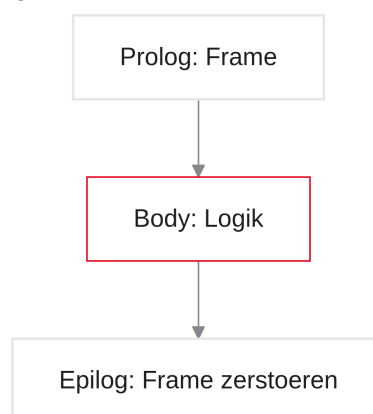
Adresse (relativ)	Inhalt
alte <i>sp</i>	Caller-Frame
12 (<i>sp</i>)	<i>ra</i>
8 (<i>sp</i>)	<i>s1</i>
4 (<i>sp</i>)	<i>s0</i>
0 (<i>sp</i>)	Padding, neue <i>sp</i>

Notizen

Pop: *lw* von 4, 8 und 12, dann *addi sp, sp, 16*. Der Bereich ab der alten *sp* gehört dem Caller.

Stack Frames (Prolog und Epilog)

- Multi-Push in standardisierte Schablonen gießen
- Burger-Metapher: Aufbau einer professionellen Funktion
- Prolog: Frame erschaffen
- Epilog: aufräumen und schließen
- Leaf vs. Non-Leaf Funktionen



Notizen

Jeder C-Compiler baut Funktionen dreiteilig: Prolog, Body, Epilog. Wer die Schablone beherrscht, schreibt verschachtelbaren Assembler.

Die Anatomie einer Funktion

- Oberes Brötchen (Prolog): zuerst — Stack reservieren, ABI-Register sichern
- Fleisch (Body): eigentliche C-Logik — Register dürfen (dank Prolog) überschrieben werden
- Unteres Brötchen (Epilog): vor `ret` — Register wiederherstellen, Stack freigeben

Notizen

Ohne Brötchen fällt der Burger auseinander. Nie den Body ohne Prolog/Epilog liefern.

Was ist ein Stack Frame?

- Der per `addi sp, sp, -X` reservierte Bereich heißt Stack Frame (Activation Record)
- Jeder aktive Aufruf besitzt seinen privaten Frame
- Inhalt typisch: gerettete `s`-Register, `ra`, optional lokale Variablen/Arrays

"Stack Overflow": zu viele Frames gestapelt
(z. B. endlose Rekursion) – Stack trifft auf den Heap.

Notizen

Lokale Arrays erweitern den Frame (z. B. `-1000`). Stack Overflow = Stack und Heap kollidieren.

Der Prolog (Die Tür öffnen)

- Steht direkt unter dem Funktions-Label
- Zählen: benötigte `s`-Register (+ `ra` falls Non-Leaf) $\times 4$
- `sp` um diesen negativen Wert verschieben
- Register mit `sw` und steigenden Offsets sichern

```
MeineFunktion:
    addi sp, sp, -16
    sw s1, 12(sp)
    sw s0, 8(sp)
```

Notizen

Danach dürfen `s0/s1` im Body überschrieben werden — Originale ruhen im Frame.

Folie

45/61

Der Body (Die Arbeit)

- Callee-Saved sind gesichert — Body ist frei
- `s0/s1` bedenkenlos nutzen; `t0–t6` für Flüchtiges (ohne Prolog-Rettung)
- Ergebnis vor dem Epilog nach `a0`

```
# --- BODY ---  
li s0, 42  
li s1, 99  
add a0, s0, s1  
# --- BODY ENDE ---
```

Notizen

Hier rechnen, iterieren, Sensoren auswerten — ABI-Rückgabe in `a0` nicht vergessen.

Folie

46/61

Der Epilog (Aufräumen)

- Spiegelbild des Prologs, umgekehrte Reihenfolge
- `lw` aus denselben Offsets
- denselben Betrag positiv zu `sp` addieren
- erst dann `ret`

```
lw s0, 8(sp)  
lw s1, 12(sp)  
addi sp, sp, 16  
ret
```

Notizen

Wiederherstellen, Frame vernichten, Heimreise. `ret` erst nach dem Gleichgewicht von `sp`.

Die Gefahr des Off-by-One

- Prolog -16 , Epilog versehentlich $+8$
- Ein Wort „Müll“ bleibt auf dem Stack
- Caller-Offsets greifen daneben — Kettenreaktion
- Schwer zu debuggen: `ret` wirkt noch korrekt

```
addi sp, sp, 8      # zu wenig, wenn der Prolog -16 war
ret
```

Notizen

Das Gleichgewichtsgesetz verzeiht keine Tippfehler im Epilog. Der Caller liest Müll und stürzt oft erst später ab.

Das komplette Template

- Master-Schablone für das Labor
- Prolog und Epilog am besten sofort paarweise tippen

```
MeineTolleFunktion:
    addi sp, sp, -16
    sw s0, 12(sp)

    # Body

    lw s0, 12(sp)
    addi sp, sp, 16
    ret
```

Notizen

Dreiklang einüben: erst Prolog+Epilog skizzieren, dann Body füllen — verhindert Ungleichgewichte.

Die Leaf-Funktion

- Leaf: ruft keine andere Funktion auf (Blatt am Baum)
- Vorteil: `ra` muss nicht auf dem Stack gesichert werden
- Kein `jal` im Body — niemand überschreibt `ra`

```
int addiere(int a, int b) {  
    return a + b;  
}
```

Notizen

Leaf-Funktionen sind günstig: oft nur `s`-Register im Prolog, kein `ra`-Backup.

Non-Leaf-Funktion

- Non-Leaf: ruft selbst weitere Funktionen auf
- Jedes `jal` im Body überschreibt `ra` mit neuer Brotkrumenspur
- Eigenes Rückflugticket zum Caller würde sonst verloren gehen
- Pflicht: `ra` im Prolog auf den Stack pushen

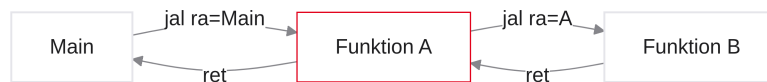
```
int berechneDistanz(int x, int y) {  
    int z = x*x + y*y;  
    return sqrt(z);    // jal ueberschreibt ra!  
}
```

Notizen

Ohne `ra` auf dem Stack endet `ret` nach dem inneren Aufruf an der falschen Adresse. Non-Leaf: `ra` immer retten.

Verschachtelte Funktionen (Nested Calls)

- Alltag: `Main` ruft `A` auf, `A` ruft `B` auf
- Sichern und Wiederherstellen von `ra`
- Mehrere Stack Frames übereinander im RAM
- Stack-Trace Schritt für Schritt visualisieren



Notizen

Nested Calls: Brotkrumenspuren beliebig tief stapeln, ohne dass sie sich stören — genau dafür existiert der Stack.

Der Nested Call (Das Szenario)

- In Hochsprachen rufen Funktionen oft Hilfsfunktionen auf
- Das heißt verschachtelter Aufruf (Nested Call)
- Szenario: `main()` ruft `calc()` auf
- `calc()` ruft intern `sum()` auf
- Zwei `jal`-Befehle, zeitlich versetzt — bevor ein `ret` folgt

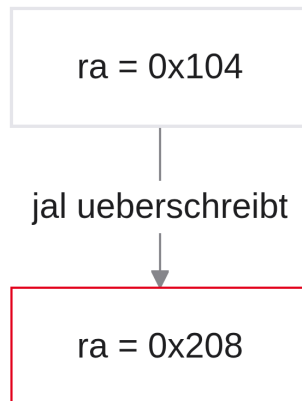
```
void main() {  
    calc(); // Aufruf 1  
}  
void calc() {  
    sum(); // Aufruf 2  
}
```

Notizen

Alltag: `Main` ruft `Calc`, `Calc` ruft `Sum`. Hardware: zwei `jal`, bevor das erste `ret` kommt.

Der Todesstoß für `ra` (Der Trace)

- Schritt 1: `Main` macht `jal ra, Calc` — `ra` = z. B. `0x104` (zurück zu `Main`)
- Schritt 2: Code in `Calc` bis zum Aufruf von `Sum`
- Schritt 3: `Calc` macht `jal ra, Sum`
- Katastrophe: Hardware überschreibt `ra` mit z. B. `0x208` — `Main`-Adresse ist weg



Notizen

Ein Register speichert nur einen Wert. Der zweite `jal` schreddert das Rückflugticket nach `Main`. Ohne Backup ist der Weg verloren.

Die Endlosschleife (Der Crash)

- `Sum` ruft `ret` auf — `ra` zeigt auf `Calc` (`0x208`): erster Rücksprung klappt
- `Calc` ist fertig und ruft `ret` auf
- In `ra` steht noch `0x208` (mitten in `Calc`)
- `Calc` springt zu sich selbst — Endlosschleife

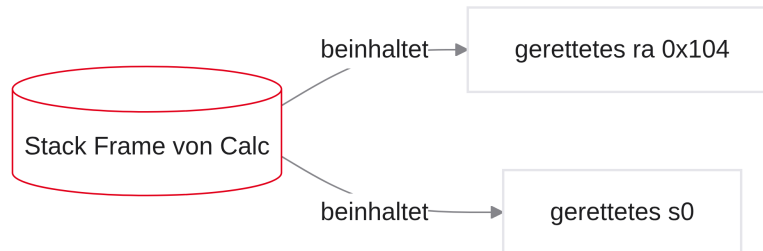
```
Calc_Zeile_X:
    jal ra, Sum      # ra wird Calc_Zeile_X+4
Calc_Zeile_Y:
    ret              # springt wieder zu Zeile_Y!
```

Notizen

`Sum` kehrt korrekt nach `Calc` zurück. Das zweite `ret` liest dasselbe `ra` und landet erneut in `Calc` — nie bei `Main`.

Die Lösung: `ra` retten

- Laut ABI ist `ra` Caller-Saved (flüchtig)
- Eine Non-Leaf-Funktion ist Callee und Caller zugleich
- Regel: `ra` einmal im Prolog des Non-Leaf sichern. Weitere innere Aufrufe nutzen dasselbe Backup, sofern der Frame stehen bleibt.
- `ra` wandert typisch als oberstes Element in den Stack Frame



Notizen

Vor dem inneren `jal`: Kopie von `ra` in den Frame. Überschreiben durch Hardware ist dann egal — das Backup liegt im RAM.

Prolog für Non-Leafs

- `ra` einmal im Prolog sichern, im Epilog wiederherstellen
- Nicht vor jedem inneren `jal` erneut, solange derselbe Frame gilt
- Ein Tailcall kann die Sicherung entbehrlich machen; das ist die Ausnahme

```
Calc:
    addi sp, sp, -16
    sw ra, 12(sp)
    sw s0, 8(sp)
```

Notizen

Minimal größerer Frame: `ra` oben, `s0` darunter. Beliebige Hilfsaufrufe im Body — Weg nach Main bleibt im RAM.

Epilog für Non-Leafs

- Spiegelverkehrt zum Prolog: zuerst `s0`, dann `ra` aus dem Frame
- `addi sp, sp, 16` gibt den Frame frei
- Erst danach `ret` — liest das wiederhergestellte `ra` (zurück zu Main)

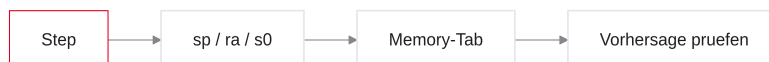
```
lw s0, 8(sp)
lw ra, 12(sp)
addi sp, sp, 16
ret
```

Notizen

Reihenfolge beachten: `ra` vor dem Freigeben des Frames laden. Dann zeigt `ret` wieder auf Main — Nested Call sicher.

Praxis: Stack-Traces in Venus

- Transfer: Nested Calls im Simulator beobachten
- Reiter Memory: Stack-Frames als Hex-Dump sehen
- Register-Ansicht: `sp`, `ra`, `s0` parallel verfolgen
- Nur Step nutzen — Run verwischt den Trace
- Ziel: jeden Push/Pop und jedes `jal/ret` vorhersagen



Notizen

Im Labor: Nested-Beispiel aus den Folien tippen. Vor jedem Step laut vorhersagen, was `ra` und `sp` tun.
Memory-Tab: Frame von `Calc` mit gerettetem `ra` sichtbar machen.

Debugging-Checkliste (Funktionen)

- Wächst `sp` im Prolog um die erwartete Byte-Zahl nach unten?
- Liegt `ra` bei Non-Leafs wirklich im Frame, bevor das innere `jal` kommt?
- Stellt der Epilog dieselben Offsets wieder her und addiert denselben Betrag?
- Nach `ret`: landet der PC bei Main — nicht erneut in `Calc`?
- Caller-Saved (`t*`) nach `jal` nicht weiterverwenden ohne Restore

Notizen

Klassiker: Epilog `+4` statt `+8`, oder `ra` nicht gerettet. Beides führt zu Endlosschleife oder Caller-Crash — Step und Memory-Tab entlarven das schnell.

Ausblick: Rekursion

- Rekursion = Funktion ruft sich selbst auf (Non-Leaf auf sich selbst)
- Jeder Aufruf braucht einen eigenen Stack Frame (frisches `ra`, frische Locals)
- Genau deshalb wächst der Stack dynamisch — feste Adresse `0x1000` würde kollabieren
- Gefahr: zu tiefe Rekursion → Stack Overflow (Stack trifft Heap)

```
int fakt(int n) {  
    if (n <= 1) return 1;  
    return n * fakt(n - 1); // Nested Call auf sich selbst  
}
```

Notizen

Die Nested-Call-Mechanik ist die Grundlage der Rekursion: jedes `fakt(n)` rettet sein `ra` und seinen Parameter-Frame, bevor es `fakt(n-1)` aufruft. Vertiefung und Laborübung folgen im Praktikum — heute reicht das Konzept.

Zusammenfassung und Ausblick

- Funktionen: `jal / ret`, Argumente und Rückgabe über `a0–a7`
- ABI: Caller-Saved vs. Callee-Saved; Non-Leafs retten `ra`
- Stack: LIFO, `sp` nach unten, Prolog/Epilog im Gleichgewicht
- Venus: Stack-Traces steppen; Rekursion = Nested Calls auf sich selbst
- Als Nächstes: Datenpfad und Pipeline visuell in Ripes

Notizen

Damit schließt Einheit 4. Im Labor Nested Calls und Frames üben. Den visuellen Datenpfad und die Pipeline betrachten wir später in Ripes.